

Docker MongoDB 설치 교본

Rocky Linux + Docker Compose · MongoDB Community · 초보자용

목표: MongoDB Community 공식 컨테이너 이미지를 Docker Compose로 설치하고, 설정파일 분리, 데이터 영구저장, 관리자 계정과 devuser 애플리케이션 계정, 27017 내부망 접속, MongoDB Compass/DBeaiver 연결까지 구성함.

1. MongoDB와 관계형 DB의 차이

구분	MariaDB/PostgreSQL/Oracle	MongoDB
구조	DB → Table → Row	DB → Collection → Document
데이터 형식	행/열	JSON 문서
기본 포트	3306/5432/1521	27017
대표 CLI	mariadb/psql/sqlplus	mongosh

MongoDB에서는 테이블 대신 collection, 행 대신 document라는 용어를 사용함.

2. 최종 디렉터리 구조

```
/home/devuser/docker/mongodb/
├── compose.yml
├── .env
├── config/
│   └── mongod.conf
├── init/
│   └── 01-init.js
└── data/
    └── ← 실제 MongoDB 데이터 영구 저장
```

MongoDB 공식 Community 이미지의 데이터 경로는 /data/db 이므로 이를 ~/docker/mongodb/data에 bind mount하여 영구 저장함.

3. Docker 확인

```
docker --version
docker compose version
sudo systemctl status docker
```

```
sudo systemctl enable --now docker
```

4. 기존 MongoDB와 27017 확인

```
rpm -qa | grep -Ei 'mongodb|mongosh'
sudo ss -lnpt | grep 27017
docker ps -a
```

기존 MongoDB가 있으면 필요한 데이터를 먼저 백업한 뒤 포트 충돌 여부를 확인함.

5. 디렉터리 생성

```
mkdir -p ~/docker/mongodb/{data,config,init}
cd ~/docker/mongodb
pwd
ls -la
```

bash

6. .env 환경설정

```
nano .env
```

bash

```
MONGO_ROOT_USERNAME=mongoadmin
MONGO_ROOT_PASSWORD=본인이정한강력한관리자비밀번호
MONGO_DATABASE=wizdb
MONGO_APP_USERNAME=devuser
MONGO_APP_PASSWORD=본인이정한devuser비밀번호
MONGO_PORT=27017
```

env

```
chmod 600 .env
```

bash

관리자 계정과 애플리케이션용 devuser 계정을 분리하는 구조임. 비밀번호는 충분히 강하게 설정함.

7. mongod.conf 만들기

```
nano config/mongod.conf
```

bash

```
storage:
  dbPath: /data/db

net:
  port: 27017
  bindIpAll: true

security:
  authorization: enabled
```

yaml

bindIpAll은 모든 인터페이스에서 listener가 대기하도록 함. 실제 접근 범위는 Rocky Linux 방화벽으로 192.168.0.0/24 등 필요한 네트워크에 제한함.

8. devuser 초기화 스크립트

```
nano init/01-init.js
```

bash

```
db = db.getSiblingDB('wizdb');

db.createUser({
  user: 'devuser',
  pwd: '본인이정한devuser비밀번호',
  roles: [
    { role: 'readWrite', db: 'wizdb' }
  ]
});

db.createCollection('users');

db.users.insertOne({
  username: 'hong',
  name: '홍길동',
  email: 'hong@test.com',
  phone: '010-3333-4444',
  role: 'USER',
  enabled: true,
  createdAt: new Date()
});
```

javascript

주의: 위 초기화 파일의 devuser 비밀번호는 .env의 MONGO_APP_PASSWORD와 동일하게 직접 맞춰야 함. docker-entrypoint-initdb.d 스크립트는 data가 비어 있는 최초 초기화 때만 실행됨.

9. compose.yml 만들기

```
nano compose.yml
```

bash

```

services:
  mongodb:
    image: mongodb/mongodb-community-server:8.0-ubi8
    container_name: wiz-mongodb
    restart: unless-stopped

    environment:
      MONGODB_INITDB_ROOT_USERNAME: ${MONGO_ROOT_USERNAME}
      MONGODB_INITDB_ROOT_PASSWORD: ${MONGO_ROOT_PASSWORD}

    ports:
      - "${MONGO_PORT}:27017"

    volumes:
      - ./data:/data/db:Z
      - ./config/mongod.conf:/etc/mongod.conf:ro,Z
      - ./init:/docker-entrypoint-initdb.d:ro,Z

    command: ["mongod", "--config", "/etc/mongod.conf"]

    healthcheck:
      test: ["CMD-SHELL", "mongosh --quiet --eval 'db.runCommand({ ping: 1 }).ok' mongodb://$MONGODB_INITDB_ROOT_USERNAME:$MONGODB_INITDB_ROOT_PASSWORD@localhost:27017/admin?authSource=admin | grep 1"]
      interval: 15s
      timeout: 10s
      retries: 10
      start_period: 30s

```

Rocky Linux의 SELinux 환경을 고려해 bind mount에 :Z를 사용함. MongoDB 공식 문서는 Community Docker 이미지를 제공하며 기본 데이터 경로로 /data/db를 사용함.

10. Compose 문법 검사

```
docker compose config
```

bash

오류가 없으면 다음 단계로 진행함. YAML에서는 TAB 대신 공백을 사용함.

11. 이미지 다운로드 및 실행

```

docker compose pull
docker compose up -d
docker ps

```

bash

처음에는 이미지를 내려받고 MongoDB 데이터 디렉터리를 초기화하므로 잠시 기다림.

12. 로그와 포트 확인

```

docker logs wiz-mongodb
sudo ss -lntp | grep 27017

```

bash

정상이라면 27017 포트에서 docker-proxy가 LISTEN 중임.

13. 관리자 계정으로 mongosh 접속

```
docker exec -it wiz-mongodb mongosh -u mongoadmin -p --authenticationDatabase admin bash
```

비밀번호를 묻는 경우 .env의 MONGO_ROOT_PASSWORD를 입력함.

```
show dbs
use admin
db.getUsers()
exit
```

javascript

14. devuser으로 wizdb 접속

```
docker exec -it wiz-mongodb mongosh "mongodb://devuser@localhost:27017/wizdb?authSource=wizdb" --password bash
```

비밀번호를 입력하면 wizdb에 접속함.

15. MongoDB 기본 명령

```
show dbs
use wizdb
show collections
db.users.find()
db.users.find().pretty()
db.users.countDocuments({})
db.users.findOne({ username: "hong" })
```

javascript

16. users 컬렉션 직접 만들기

```
use wizdb

db.createCollection("users")

db.users.insertOne({
  username: "kim",
  name: "김철수",
  email: "kim@test.com",
  phone: "010-1111-2222",
  role: "ADMIN",
  enabled: true,
  createdAt: new Date()
})
```

javascript

17. CRUD 연습

```
// CREATE
db.users.insertOne({
  username: "lee",
  name: "이순신",
  email: "lee@test.com",
  role: "USER",
  enabled: true
})

// READ
db.users.find()
db.users.findOne({ username: "lee" })

// UPDATE
db.users.updateOne(
  { username: "lee" },
  { $set: { email: "lee2@test.com" } }
)

// DELETE
db.users.deleteOne({ username: "lee" })
```

text

18. devuser 권한 확인

```
use wizdb
db.getUser("devuser")
```

javascript

현재 예제에서는 devuser에 wizdb의 readWrite 권한만 부여함. 애플리케이션 계정에는 root 권한보다 필요한 DB의 readWrite 정도를 부여하는 것이 안전함.

19. Rocky Linux 방화벽

내부망 192.168.0.0/24에서만 MongoDB 27017 접속을 허용하는 예임.

```
sudo firewall-cmd --permanent --remove-port=27017/tcp

sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source address="192.168.0.0/24" port port="27017" protocol="tcp" accept'

sudo firewall-cmd --reload
sudo firewall-cmd --list-all
```

bash

20. Windows에서 포트 테스트

현재 서버 내부 IP SERVER_IP 기준:

```
Test-NetConnection SERVER_IP -Port 27017
```

powershell

TcpTestSucceeded : True이면 Windows에서 Docker MongoDB 포트까지 연결됨.

21. MongoDB Compass 연결

MongoDB GUI로는 MongoDB Compass를 사용하면 편리함. devuser은 wizdb에 인증되는 계정이므로 authSource=wizdb를 지정함.

```
mongodb://devuser:비밀번호@SERVER_IP:27017/wizdb?authSource=wizdb
```

text

관리자 접속은 authSource=admin을 사용함.

```
mongodb://mongoadmin:관리자비밀번호@SERVER_IP:27017/admin?authSource=admin
```

text

22. DBeaver 연결

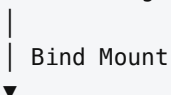
DBeaver의 MongoDB 지원 범위는 사용하는 에디션/드라이버에 따라 달라질 수 있음. MongoDB 전용 관리에는 Compass가 가장 직관적이며, DBeaver에서 MongoDB 연결 옵션이 제공되는 환경이라면 같은 Host/Port/사용자/인증 DB 정보를 사용함.

항목	값
Host	SERVER_IP
Port	27017
Database	wizdb
Username	devuser
Authentication DB	wizdb

23. 데이터 영구저장 원리

```
Rocky Linux
/home/devuser/docker/mongodb/data
```

text



```
MongoDB Container
/data/db
```

docker compose down을 실행해도 ~/docker/mongodb/data는 남음. 다시 docker compose up -d 하면 기존 데이터베이스를 사용함.

주의:

```
~/docker/mongodb/data는 실제 DB 원본이므로 함부로 삭제하지 않음.
```

text

24. 자주 사용하는 Docker 명령

목적	명령
설정 검사	docker compose config
이미지 다운로드	docker compose pull

목적	명령
시작	<code>docker compose up -d</code>
상태	<code>docker ps</code>
중지	<code>docker compose stop</code>
재시작	<code>docker compose restart</code>
컨테이너 제거	<code>docker compose down</code>
로그	<code>docker logs wiz-mongodb</code>
실시간 로그	<code>docker logs -f wiz-mongodb</code>
관리자 접속	<code>docker exec -it wiz-mongodb mongosh -u mongoadmin -p --authenticationDatabase admin</code>

25. 백업과 복구 기초

MongoDB의 논리 백업은 `mongodump`, 복구는 `mongorestore`를 사용함. 운영 데이터는 단순히 실행 중인 data 디렉터리를 복사하는 방식만 믿지 말고 공식 백업 도구 또는 적절한 스냅샷 전략을 사용함.

```
mkdir -p ~/mongodb-backup bash

docker exec wiz-mongodb mongodump --username mongoadmin --password '관리자비밀번호' --authenticationDatabase admin --db wizdb --out /tmp/backup
```

비밀번호를 명령행에 직접 쓰면 shell history 등에 남을 수 있으므로 실제 운영 자동화에서는 자격증명 노출을 피하는 방식을 사용함.

26. 초보자가 기억할 핵심 명령

```
cd ~/docker/mongodb bash
docker compose config
docker compose up -d
docker ps
docker logs wiz-mongodb
sudo ss -lntp | grep 27017
docker exec -it wiz-mongodb mongosh -u mongoadmin -p --authenticationDatabase admin
docker compose down
```

27. MariaDB·PostgreSQL·Oracle·MongoDB 비교

DB	포트	CLI	데이터 개념	영구저장 컨테이너 경로
MariaDB	3306	mariadb	Table/Row	/var/lib/mysql
PostgreSQL	5432	psql	Table/Row	/var/lib/postgresql/data
Oracle	1521	sqlplus	Table/Row	/opt/oracle/oradata
MongoDB	27017	mongosh	Collection/Document	/data/db

28. 최종 점검표

점검 항목	정상 기준
Compose	docker compose config 오류 없음
컨테이너	wiz-mongodb 실행 중
영구저장	~/docker/mongodb/data 사용
Listener	27017 LISTEN
관리자	mongoadmin 로그인 가능
DB	wizdb 사용 가능
앱 사용자	devuser 로그인 가능
권한	devuser → wizdb readWrite
방화벽	192.168.0.0/24 → 27017 허용
Windows	TcpTestSucceeded : True
GUI	MongoDB Compass 연결 가능

29. 보안상 꼭 기억할 점

MongoDB 27017 포트를 인터넷 전체에 그대로 공개하지 않는 것을 권장함. 인증을 활성화하고 방화벽으로 필요한 내부망 또는 애플리케이션 서버만 허용함. 관리자 계정과 애플리케이션 계정을 분리하고, 애플리케이션에는 최소 권한을 부여함.

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME...)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.